Natural Language to SQL Business Intelligence System

Reasoning-Model Inference and Chain-of-Thought Suppression

Mit Mhatre — 16014223052
Gurjit Singh — 16014223037

Technical Report

`Stack:` React 18 + TypeScript · FastAPI · Groq · openai/gpt-oss-20b · postgresql · ChromaDB

`Measured:` 472 ms mean model latency · 1,231 ms mean end-to-end · 0 reasoning markers observed

Abstract

Natural-language business-intelligence (NL-BI) systems translate plain-English queries into executable SQL, evaluate them against an analytical data store, and return structured visual and narrative responses. This report presents the design and implementation of such a system, built upon a decoupled architecture comprising a React 18 + TypeScript single-page application (SPA), a FastAPI REST backend, an isolated LLM service layer, and hosted inference on Groq using the reasoning-capable model `openai/gpt-oss-20b`.

Three principal concerns informed the architectural decisions. First, the previous monolithic interface conflated presentation, orchestration, and data access, rendering the frontend tier a credential holder and preventing independent deployment. Second, local model serving imposed latency characteristics inappropriate for an interactive analytics tool, with request timeouts provisioned for CPU-scale inference on the order of minutes. Third, the selected replacement model is reasoning-capable: it produces internal deliberation on a channel structurally distinct from its answer, and naive integration risks rendering that deliberation to end users.

The proposed system introduces a provider-agnostic LLM abstraction backed by an asynchronous Groq implementation; an end-to-end `async`/`await` request pipeline with explicit thread offloading for blocking operations; and a four-layer, defence-in-depth reasoning-suppression mechanism. The React frontend communicates exclusively through typed REST calls and holds no credentials or provider references. Measured over the deployed system, mean model latency is 472 ms, mean end-to-end API latency is 1,231 ms, and the initial JavaScript payload is 55.3 kB (gzipped) following chart-library code-splitting. Thirty-six automated tests validate the reasoning-leakage and provider-behaviour properties; no reasoning marker, credential, connection string, or stack trace was observed in any API response or compiled client bundle.

Keywords: natural-language-to-SQL, business intelligence, FastAPI, React, hosted inference, Groq, reasoning models, chain-of-thought suppression, service-layer architecture, API security.

1 Introduction

1.1 Problem Context

Text-to-SQL represents one of the most commercially consequential applications of large language models: given a question posed by a non-technical stakeholder, the system generates an executable query over a governed relational schema, eliminating the analyst-in-the-loop step that traditionally separates a business question from its answer [1]. The system described herein implements this pipeline end-to-end, encompassing schema retrieval by embedding similarity, schema-grounded prompting, multi-layer SQL validation, read-only execution, automatic chart selection, and result summarisation.

The core pipeline was functionally sound. What had become inadequate was the surrounding architecture: the user-facing interface, the model serving infrastructure, and the coupling between the two.

1.2 Motivation for a Decoupled Frontend

The original interface was implemented as a monolithic Python application. This architecture imposed three critical constraints. First, there was no separation of tiers: the dashboard module instantiated its own database engine directly and executed raw SQL from the presentation layer, making the frontend tier a credential holder. Second, no independent evolution was possible, as frontend and backend shared a unified dependency set and deployment unit. Third, the interaction model was limited, with no route structure, no client-side state model, and no mechanism for partial or optimistic UI states.

Separating the frontend from the backend via a REST contract resolves all three concerns: the browser becomes a pure consumer of typed JSON, the API becomes the sole credential holder, and each tier can be built, tested, versioned, and deployed independently [2].

1.3 Motivation for Promoting the API Layer

FastAPI [3] was already present in the project but underutilised. Promoting it to the authoritative system boundary confers three properties the previous architecture lacked: declarative request validation and response modelling via Pydantic [15], ensuring a machine-checked contract in both directions; native ASGI concurrency [4], enabling LLM calls (which are almost entirely I/O-bound) to be awaited rather than blocking a worker thread; and automatic OpenAPI schema generation, giving the TypeScript frontend a single authoritative source for its interface types.

1.4 Motivation for Hosted Inference

Local model serving offers real advantages: no per-token cost, no data egress, and no external dependency. However, it also carries an operational cost profile that becomes prohibitive outside development. The prior implementation configured a 300-second request timeout with an explicit comment noting CPU inference, and held the model resident in accelerator memory across requests. These are the timeout constants of a system designed around the possibility of minute-scale response times.

This profile forces a specific deployment topology: every environment must also provision a model server, allocate accelerator memory, and pre-fetch a multi-gigabyte model artefact. Horizontal scaling multiplies this cost per replica. Hosted inference inverts the arrangement: the backend becomes a stateless service that holds a credential and makes an HTTPS call, and concurrency is bounded by provider rate limits rather than by local hardware.

1.5 Motivation for a Reasoning-Capable Model with Private Reasoning

The target model, `openai/gpt-oss-20b`, is an open-weight reasoning model [8]. Reasoning models improve accuracy on multi-step problems by generating an extended intermediate derivation before committing to an answer [5]. For SQL generation over a wide schema, this is precisely the category of task that benefits: the model must resolve which tables are relevant, identify the join keys that connect them, and determine the aggregation implied by the question.

The same capability creates a disclosure risk. The intermediate derivation is not the answer. It is verbose, may contain discarded hypotheses, and frequently restates schema details, prompt instructions, and internal heuristics not intended for publication. A system that surfaces this reasoning degrades the user experience and widens its disclosure surface. The engineering objective is therefore to *benefit* from reasoning while *never* exposing it.

1.6 Contributions

This report documents the following contributions:

1. A provider-agnostic LLM abstraction and its Groq implementation (§3, §6);
2. An end-to-end asynchronous request pipeline with explicit thread offloading for blocking operations (§5);
3. A four-layer, defence-in-depth mechanism for suppressing model reasoning, with a regression suite encoding each known leakage shape (§7);
4. A React frontend built to a typed contract, holding no credentials or provider references (§4);
5. A security analysis of the resulting trust boundaries (§8) and a measured performance evaluation (§9, §11).

2 Previous Architecture

2.1 Component Topology

The previous system comprised three processes. A Python presentation application communicated with a FastAPI backend over HTTP for chat queries. However, the dashboard module within that same presentation layer bypassed the API entirely, constructing its own database engine from environment credentials and executing raw SQL directly. A locally-managed Ollama daemon served the language model on `localhost:11434`. Two of the three processes therefore held database credentials, and one required operator-managed model serving.

2.2 Interface Limitations

The presentation model re-executed the full application script on every user interaction, reconstructing the widget tree from module-level state. This produced several concrete deficiencies. Conversation history was stored in a session object and required re-instantiating all prior visualisations on every interaction, producing work proportional to conversation length. Chart rendering logic was duplicated: the backend produced a Plotly figure, and a separate client-side fallback re-derived column types and chart selection independently, with no shared tests. The layout had no responsive breakpoints, and accessibility affordances such as ARIA roles, focus management, and keyboard semantics were absent.

2.3 Local Inference Limitations

The Ollama client was synchronous. Because route handlers declared `async` functions but called a blocking

HTTP client internally, concurrent requests serialised on the event loop. Operationally, every deployment required an Ollama installation, a resident model artefact, and sufficient accelerator memory. The health endpoint exposed the provider name as a first-class field of the public API contract.

2.4 What Was Retained

The migration deliberately preserved the entire domain pipeline. Schema extraction, ChromaDB embedding retrieval [6, 7], the few-shot SQL template library, the eight-layer SQL validator, the confidence scorer, the read-only executor, the auto-retry engine, the chart-selection engine, and the insight generator are unchanged in behaviour. The migration is architectural, not functional.

3 Proposed Architecture

3.1 Layer Diagram

The proposed system is structured across five tiers, as illustrated in Figure 1. The browser tier (React 18 SPA) communicates exclusively with the FastAPI tier over `/api/*` on its own origin. FastAPI delegates LLM work to a service layer, which invokes a concrete provider implementation through a two-method abstract contract. The provider communicates with Groq over HTTPS using a server-held Bearer token.

Tier 1 — Browser (untrusted)
React 18 + TypeScript SPA, Vite build
no credentials, no provider URL, no DB access

↓ HTTPS · JSON · same-origin `/api/*`

Tier 2 — FastAPI (trusted)
`routes.py` · `schemas.py` · `serialization.py`

↓ service call

Tier 3 — LLM Service Layer
`service.py` · `sanitizer.py` · `prompts.py`

↓ LLMProvider contract (`base.py`)

Tier 4 — Provider
`groq_provider.py` — async httpx, retries, error taxonomy

↓ HTTPS + Bearer token (server-held)

Tier 5 — Groq · `openai/gpt-oss-20b`
returns final answer only

Figure 1: Five-tier system architecture. Each tier knows only the tier immediately below it. The browser holds no secrets; all provider communication originates from Tier 4.

3.2 Request Lifecycle

A `POST /api/query` request traverses ten stages: (1) declarative validation via `QueryRequest` Pydantic model; (2) schema context retrieval via ChromaDB embedding similarity, dispatched to a thread pool (CPU-bound); (3) prompt assembly by the service layer; (4) asynchronous LLM generation via `GroqProvider`; (5) eight-layer SQL validation; (6) confidence scoring; (7) read-only SQL execution, dispatched to a thread pool; (8) self-correction via the async retry engine, if execution fails; (9) chart selection and Plotly figure construction, dispatched to a thread pool; (10) serialisation of the declared `QueryResponse` fields.

3.3 Separation of Concerns

The load-bearing architectural decision is that no layer imports from more than one level below it. `routes.py` imports `llm_service`; it has no knowledge that Groq exists. `service.py` calls `get_llm_provider()`; it does not know the provider is HTTP-based. `groq_provider.py` knows about HTTP, Bearer tokens, and Groq's response schema, and knows nothing of SQL, schemas, or business intelligence. Adding a second provider requires writing one class, registering it, and setting a configuration variable. Nothing else changes.

4 Frontend Architecture

4.1 Framework Selection

React 18 [12] was selected for three reasons specific to this project. The backend emits Plotly figure JSON; React's `useRef` + `useEffect` idiom maps cleanly onto the imperative DOM mounting and teardown that Plotly requires, allowing a lightweight local wrapper rather than a third-party integration package that would constrain React's version. TypeScript [14] enables the Pydantic API models to be mirrored as interfaces (`src/api/types.ts`) checked by the compiler at every call site. Finally, the application has two views and one conversation thread, a scope that requires no router, state-management library, or data-fetching library, keeping the dependency surface minimal.

4.2 Component Architecture

The component tree is organised as follows:

- **App** — view routing, cross-cutting data
  - **Sidebar** — navigation, health, metrics, suggestion chips
  - **AssistantView** — conversation surface
    - **EmptyState** — onboarding and starter questions
    - **AnswerCard** (per turn) — **ConfidenceBadge**, **PlotlyChart** (lazy), **DataTable**
    - **Composer** — auto-growing textarea with cancel control
  - **DashboardView** — KPI row and six analytics panels

Components are presentational and receive data exclu-

sively through props. All data fetching resides in custom hooks. `AnswerCard` is the sole component that renders model-derived content, and it can render only the fields declared by `QueryResponse`.

4.3 State Management

Three categories of state are managed by three mechanisms, without an external library. Conversation state is owned by `useAssistant`, which maintains the message list, a single in-flight `AbortController`, the loading flag, and the session identifier; cancellation is first-class. Server resource state is managed by `useApiResource`, a 55-line hook providing fetch-on-mount, abort-on-unmount, error capture, and manual reload. View state (active view, mobile drawer open/closed) is managed with `useState` in `App`. Session identity persists in `sessionStorage` behind a `try`/`catch`, so private-browsing modes that prohibit storage access degrade gracefully to an in-memory identifier.

4.4 API Communication

All network access is centralised in `src/api/client.ts`, which exposes five typed functions and a generic `request<T>` helper. The helper prefixes every path with `VITE_API_BASE_URL` (defaulting to `/api`, proxied by Vite in development); maps transport failures to a synthetic `network_error`; parses the uniform error envelope into a typed `ApiError`; and re-raises `AbortError` unchanged so cancellation is not misreported as failure. Because there is exactly one egress point, the property "the browser never contacts the model provider" is verifiable by reading one file.

4.5 Responsive Design and Accessibility

A single dark design system is expressed as CSS custom properties across three breakpoints (1024 px, 900 px, 560 px). At 900 px and below, the sidebar converts to a fixed-position drawer with a scrim overlay and Escape-to-close behaviour. Wide content (tables, charts) scrolls within its own container; the page body never scrolls horizontally.

Accessibility provisions include: a skip link; `role="log"` with `aria-live="polite"` on the conversation container; semantic `<table>` markup with scoped headers and an off-screen caption for data tables; ARIA tab semantics on result view tabs; `aria-label` on icon-only controls; a visible focus ring on every interactive element; and a `prefers-reduced-motion` block that disables all animations.

4.6 Client-Side Security

The frontend holds no secrets, and this is structural rather than aspirational. Vite inlines only `VITE_`-prefixed environment variables into the bundle; the Groq API key is not defined as such a variable and therefore cannot be inlined. Verification of the production bundle found no key value, no provider URL, and no database connection string. Model-derived text is inserted as text nodes and never via `dangerouslySetInnerHTML`, so model output cannot inject markup. Chart JSON is passed to Plotly as data, not evaluated.

5 FastAPI Backend Architecture

5.1 Endpoint Design

Table 1 summarises the seven API endpoints. The introduction of `GET /api/dashboard` is architecturally significant: it moves the six aggregate queries previously executed client-side into server-owned constants written in dialect-neutral SQL, allowing them to run unchanged on both PostgreSQL and SQLite.

Table 1: API endpoint summary.

Method	Path	Purpose
POST	`/api/query`	NL → SQL → chart → insight
GET	`/api/suggestions`	Schema-aware starter questions
GET	`/api/dashboard`	Six pre-built analytics panels
GET	`/api/schema`	Cached schema metadata
GET	`/api/history`	Session conversation turns
GET	`/api/health`	Schema and provider readiness
GET	`/api/metrics`	Aggregate performance metrics

5.2 Request Validation and Response Models

Validation is declarative and total. `QueryRequest` bounds question length (3–500 characters), collapses whitespace, and restricts the session identifier to `[A-Za-z0-9_-]{1,64}` — a character allow-list that prevents the identifier from carrying prompt content, since it is used as a dictionary key in prompt context construction.

Every endpoint declares `response_model`; FastAPI serialises only declared fields. This is a structural containment property: even if an upstream object were to acquire a field carrying provider internals or model reasoning, it could not be serialised without an explicit schema change. `GenerationInfo` declares five scalar fields and no field capable of carrying model text.

5.3 Asynchronous Request Handling

The provider uses one shared `httpx.AsyncClient` with connection pooling (20 maximum, 10 keep-alive), created lazily under an `asyncio.Lock` and reused for

the process lifetime. All blocking operations are explicitly offloaded:

Table 2: Thread-offloading strategy for blocking operations.

Operation	Handling
Groq completion	`await` (native async)
Embedding retrieval	`asyncio.to_thread`
Template lookup	`asyncio.to_thread`
SQL execution	`asyncio.to_thread`
Chart rendering	`asyncio.to_thread`
Insight generation	`asyncio.to_thread`
Dashboard build	`asyncio.to_thread`

5.4 Error Handling

A three-level scheme produces a uniform envelope `{"error": code, "message": text}`. Typed `LLMError` subclasses carry an HTTP status and stable error code; route handlers map them mechanically (e.g., a 429 from the provider becomes `llm_rate_limited` with HTTP 429). Application errors use the same envelope shape. A global handler for unhandled exceptions logs the traceback server-side and returns a generic client message, preventing any internal detail from reaching the browser.

5.5 Configuration Management

Configuration is a single Pydantic Settings object loaded from environment variables and an optional git-ignored `.env` file. Secrets have no default value: `groq_api_key` defaults to the empty string, `llm_configured` reports its presence, and the provider raises `LLMConfigurationError` on an unauthenticated call attempt. CORS was tightened from wildcard origins and methods to an explicit origin allow-list, `[GET, POST, OPTIONS]`, and `[Content-Type]`, with `allow_credentials=False`.

6 LLM Migration

6.1 Migration Rationale

Table 3 summarises the principal differences between the prior local inference configuration and the current hosted configuration. The measured model latency of 472 ms mean in the migrated system contrasts with a prior timeout of 300 s in two places, with an explanatory comment about CPU inference, describing a qualitatively different operating regime.

Table 3: Comparison of local and hosted inference configurations.

Dimension	Local (prior)	Hosted (current)
Placement	Operator daemon, localhost	Managed HTTPS endpoint
Model artefact	Pulled locally (multi-GB)	Provider-side
Accelerator	Required per replica	None required
Client	Synchronous blocking	Async, pooled
Timeout	300 s	60 s (configurable)
Error taxonomy	Untyped strings	Typed errors → HTTP status
Scaling unit	Model replica	Stateless API replica
Data egress	None	Prompt leaves trust boundary

6.2 API Invocation

Groq exposes an OpenAI-compatible `POST /openai/v1/chat/completions` endpoint [11]. The provider constructs a request with `reasoning_format="hidden"`, `reasoning_effort="low"`, and `stream=false`. Two details are load-bearing.

Completion budget. Reasoning tokens are billed against `max_completion_tokens`. Requesting exactly the answer budget risks the model exhausting it during deliberation. The provider therefore adds headroom scaled to the configured effort (512, 1024, or 2048 tokens for low, medium, or high effort respectively).

Parameter fallback. If a deployment rejects the reasoning parameters with HTTP 400, the provider detects this, removes the parameters, reduces the budget accordingly, and retries transparently. The system degrades to sanitiser-only suppression rather than failing.

6.3 The openai/gpt-oss-20b Model

`openai/gpt-oss-20b` is a 20-billion-parameter open-weight model in OpenAI's gpt-oss family, designed for reasoning with a configurable effort level and for agentic and structured-output use cases [8]. The `reasoning_effort="low"` setting is well-matched to schema-grounded SQL generation: the prompt already supplies schema information, foreign key declarations, few-shot exemplars, and explicit SQL construction rules, so the model's remaining work is selection and assembly rather than open-ended derivation. Empirically, this setting reduced completion tokens from 68 to 32 on an identical minimal SQL prompt.

6.4 Operational Implications

Failure modes shift from "is the daemon running?" to "is the credential valid, are we within rate limits, is the provider available?" The health endpoint verifies both credential validity and model availability by listing accessible models and confirming that `LLM_MODEL` is present on the account. Rate limiting is not theoretical: a burst of eight sequential queries during evaluation produced three HTTP 429 responses, which surfaced correctly as `llm_rate_limited` with a retry hint, not as server errors.

7 Reasoning Suppression

7.1 Why Reasoning Must Not Be Exposed

Reasoning models trained with chain-of-thought supervision emit an extended intermediate derivation before committing to a final answer [5]. The `gpt-oss` family uses the Harmony response format, in which the model writes to named channels — `analysis` for private reasoning and `final` for the user-facing answer — delimited by special tokens [9].

Three considerations prohibit surfacing this reasoning. First, the analysis channel contains discarded hypotheses and self-corrections; rendering them alongside a correct query provides a misleading signal about system reliability. Second, the reasoning channel systematically restates prompt content — schema details, internal rules, and few-shot exemplars — constituting an unintended paraphrase of the system prompt. Third, providers position reasoning as internal to the model's processing and not intended for direct user presentation.

7.2 The Four-Layer Mechanism

Reasoning suppression is enforced by four independent layers:

Layer 1 — Suppress at the source. `reasoning_format="hidden"` instructs the Groq API to omit reasoning from the response body entirely [10]. Empirical verification against the live API confirmed that with this parameter, the `reasoning` field is not transmitted. Adding `reasoning_effort="low"` further reduced completion tokens from 68 to 32 on an identical prompt.

Layer 2 — Parse narrowly. The internal `_parse_completion` method reads `choices[0].message.content` exclusively. `LLMResponse` is a frozen dataclass with no field capable of holding reasoning content. The parser records only a boolean flag indicating whether a reasoning key was present, for telemetry purposes.

Layer 3 — Sanitise defensively. `app/llm/sanitizer.py` applies a purely subtractive transformation to every string exiting the LLM layer. The sanitiser handles: balanced tag blocks (`<think>`, `<thinking>`, `<reasoning>`, `<analysis>`, and six further variants), case-insensitively and across newlines; Harmony channel markers; the detokenised spelling of Harmony special tokens; truncation artefacts (an unterminated opening tag indicates generation was cut off during deliberation, so everything from the tag onward is discarded); and Markdown reasoning headings.

Two invariants are critical. The transformation is *only ever subtractive*: it never reconstructs, summarises, or paraphrases hidden content. It *fails closed*: if sanitisation produces an empty string, the provider raises `LLMResponseError` rather than returning an empty answer. The sanitiser runs twice — once in the provider, once in the service layer — so a future provider that omits the sanitiser call cannot leak.

Layer 4 — Constrain serialisation. FastAPI response models declare their fields exhaustively (§5). Even if an internal object were to acquire a reasoning field, it could not be serialised without an explicit schema change.

7.3 Streaming

The backend does not stream. This is a deliberate safety decision: a reasoning model's token stream interleaves analysis and answer tokens; a naive relay would forward analysis tokens to the browser before the channel boundary is known. Safe streaming would require a stateful, channel-aware filter that buffers until a final-channel marker is observed — reintroducing most of the latency streaming was intended to reduce, while introducing a class of parser defect with a disclosure consequence. The request sets `stream: false` explicitly, and a test asserts this property so it cannot regress silently.

8 Security Analysis

8.1 Trust Boundaries

The architecture establishes three trust zones: the browser (untrusted), the FastAPI backend (trusted), and the provider plus database (external). Credentials flow from environment variables to the trusted zone only; the browser holds none.

8.2 API Key Protection

The Groq API key exists in exactly one place at rest (a git-ignored `.env`) and one place in transit (the `Authorization` header of a server-originated HTTPS request). It has no source-code default, is never logged,

never appears in an error payload, and is absent from the compiled client bundle. Tests assert explicitly that the key travels only in the `Authorization` header and that a 401 error message does not contain the key value.

8.3 Input Validation

The eight-layer SQL validator enforces: `SELECT`/`WITH`-only statements; a blocked keyword list; injection pattern matching; stacked statement detection; structural parse verification; `LIMIT` enforcement; table-name verification against the live schema; and subquery mutation checks. Adversarial probing during evaluation — submitting a question that combined a destructive SQL clause with an analytical request — produced a single read-only `SELECT`; the destructive clause did not survive extraction and validation.

8.4 Common Vulnerability Mitigations

Table 4 summarises the common configuration vulnerabilities this design forecloses by construction.

Table 4: Structural mitigations for common configuration vulnerabilities.

Vulnerability	Structural mitigation
API key in frontend bundle	No `VITE_`-prefixed secret defined; bundle verified
Key committed to VCS	No source default; `.env` git-ignored
Wildcard CORS	Explicit origin list from `CORS_ALLOW_ORIGINS`
Reasoning rendered in UI	Four independent suppression layers
Stack trace to client	Global handler returns a generic message
Browser holding a DB URL	Dashboard queries moved server-side
Silent startup without key	Requests fail with `503 llm_not_configured`
NaN breaking client JSON	Serialiser maps non-finite floats to `null`

9 Performance and Scalability

9.1 Asynchronous Architecture

ASGI concurrency provides meaningful benefit only when nothing blocks the event loop. The previous system violated this constraint by invoking a synchronous HTTP client inside `async` route handlers, serialising all generation under concurrency. The migrated system awaits genuine I/O and explicitly offloads every blocking operation to the thread pool (Table 2). During the approximately 470 ms a model call is outstanding, the event loop is available for other requests.

9.2 Hosted Inference

Removing local model serving removes the per-replica accelerator requirement and the associated cold-start penalty. Scaling is now horizontal over stateless replicas; the practical ceiling is the account's rate limit rather than hardware capacity. This constraint is contractual and therefore adjustable by configuration or spend rather than by infrastructure procurement.

9.3 Frontend Payload

Code-splitting the Plotly chart runtime reduces the initial JavaScript payload significantly. The chart bundle loads on demand when the user first views a chart result, keeping the cold-load experience lightweight.

10 Testing and Evaluation

10.1 Testing Strategy

Table 5 summarises the testing strategy across eight categories. The design principle for reasoning-leakage testing is *test by shape, not by sample*: enumerate the structural forms reasoning can take and assert that each is removed, rather than testing only known output samples.

Table 5: Testing strategy summary.

Category	Method	Coverage
Reasoning leakage	Unit, exhaustive by shape	25 tests
Provider behaviour	Unit, mock transport	11 tests
SQL validation	Pre-existing unit tests	validator, confidence, visualisation
API contract	Live request/response	all endpoints
Error conditions	Adversarial live requests	validation, 404, throttling, injection
Security	Bundle and response scan	keys, DB URLs, tracebacks, reasoning
UI	Browser-driven interaction	desktop and mobile viewports
Performance	Timed live queries	model, SQL, and end-to-end latency

10.2 Reasoning-Leakage Testing

The test suite enumerates: balanced tags across nine tag names; case variation; multiple and multiline blocks; unterminated opening tags; orphan closing tags; Harmony final-channel extraction; analysis-channel content without a final marker; the detokenised `assistantfinal` spelling; and stray control tokens. Complementary negative tests assert that clean SQL, clean prose, and SQL containing `<` and `>` comparison operators are returned unmodified. A final assertion checks the post-condition

directly: for every input sample, the sanitised output contains no reasoning marker.

10.3 Provider Testing

An injected `httpx.MockTransport` allows tests to exercise the real client construction against a controlled transport. Tests assert: the request shape (model, reasoning parameters, `stream: false`, budget headroom, message roles); that the API key travels only in the `Authorization` header; that a `reasoning` field in a mocked response never reaches `LLMResponse`; that a reasoning-only completion raises rather than returning; that 401/429 responses map to typed errors without echoing the key; that a 400 rejection of reasoning parameters triggers a transparent retry without them; and that the health probe reports correctly.

10.4 Coverage Limitations

The evaluation does not include automated end-to-end testing, load or soak testing, unit tests for FastAPI routes (verified live rather than in CI), accessibility audit tooling, or evaluation of SQL accuracy against a labelled benchmark such as Spider [1]. Confidence scoring is a heuristic proxy, not ground truth.

11 Results

11.1 Functional Verification

The complete request pipeline was exercised through the browser: React → FastAPI → LLM service → Groq → `openai/gpt-oss-20b` → sanitised final answer → FastAPI → React. The health endpoint reported: `status: healthy`, `provider: groq`, `reasoning_suppression: enabled`, `schema_status: 5 tables loaded`.

11.2 Latency

Eight sequential live queries were timed; five completed before provider rate throttling. Table 6 reports the decomposed latency measurements.

Table 6: Latency measurements over five completed live queries.

Metric	Mean	Median	Min	Max
Model (ms)	472	512	265	536
SQL exec. (ms)	65	86	15	96
End-to-end (ms)	1,231	1,225	868	1,620

Dashboard construction required 627–684 ms server-side for six aggregate queries over 58,917 fact rows. These measurements represent single-client observations on a free-tier account against a SQLite database; they characterise the system's order of magnitude, not a production capacity envelope. No comparable benchmark of the prior local inference configuration was collected, so the comparison to the previous system rests on its own timeout constants rather than on measured latency.

11.3 Frontend Bundle

Table 7: Frontend bundle size before and after code-splitting.

Artefact	Before	After
Initial JS	1,273 kB (434 kB gz)	**172 kB (55 kB gz)**
Chart runtime	bundled	1,098 kB (378 kB gz), on demand
CSS	17.95 kB (4.45 kB gz)	17.95 kB (4.45 kB gz)

Code-splitting the Plotly chart runtime achieves an 87% reduction in initial JavaScript payload (gzipped). TypeScript compilation passes under `strict`, `noUnusedLocals`, and `noUnusedParameters`.

11.4 Reasoning Suppression

No reasoning marker appeared in any live response. Key-by-key inspection at every depth of every response found no field named `reasoning`, `thinking`, `analysis`, `thought`, or `chain_of_thought`. Every response carried `generation.reasoning_suppressed: true`. Thirty-six automated tests cover the suppression mechanism.

11.5 Error Handling

The evaluation burst produced three HTTP 429 responses from the provider. Each surfaced as `{"error": "llm_rate_limited", "message": "Groq rate limit reached. Please retry in a few seconds."}` with HTTP 429, not as a 500 or a timeout. Adversarial inputs behaved as designed: empty or over-length questions returned 422 with field-specific messages; an injection attempt produced a single read-only `SELECT`.

11.6 Discussion

Three observations merit separation from the raw measurements.

Where the time goes changed. At 472 ms mean, generation is no longer the dominant term in end-to-end latency; roughly half of the median request is application work. This inverts the prior optimisation calculus: with minute-scale inference, no other component was worth measuring; now embedding retrieval and chart rendering are legitimate optimisation targets.

The binding constraint moved outward. The capacity ceiling was previously local hardware; it is now a provider rate limit — a contractual constraint adjustable by configuration or spend rather than by infrastructure procurement. The throttling observed in the evaluation makes this concrete.

Suppression is cheap when structural. The most effective suppression layer costs nothing at runtime: `LLMResponse` has no field capable of carrying reasoning, and response models serialise only what they declare. The regex sanitiser is the visible defence, but the containment property derives from types.

12 Limitations and Future Work

12.1 Current Limitations

Rate limiting. Demonstrated during evaluation. Mitigations (caching, backoff, higher service tier) are not yet implemented.

In-process state. Conversation memory and metrics are per-process. Multi-replica deployment would route follow-up questions inconsistently and fragment metrics.

Data egress. Prompts — including schema metadata and user questions — leave the trust boundary. This is inherent to hosted inference and represents the clearest trade-off relative to local serving.

Provider dependency. Availability is coupled to Groq. The provider abstraction limits the impact to one implementation file, but no fallback provider is implemented.

Streaming. Deliberately omitted (§7), at the cost of incremental output delivery.

SQL accuracy. No labelled-benchmark evaluation has been conducted; confidence scoring is a heuristic proxy.

Single-tenant assumptions. No authentication, authorisation, or per-user quota management is implemented.

12.2 Future Improvements

Near term. Cache schema-retrieval embeddings by question hash; add a response cache for identical questions; externalise conversation memory and metrics to Redis; add route-level tests to CI; expose rate-limit headroom on the health endpoint.

Medium term. Implement a second provider behind the existing abstract contract to validate portability and enable failover; add an automated end-to-end test suite; evaluate SQL accuracy against a labelled benchmark; add per-user authentication and quotas.

Longer term. A channel-aware streaming path, built to the buffering discipline described in §7; adaptive `reasoning_effort` selection based on estimated question complexity; and a local provider implementation for deployments where data-residency requirements prohibit prompt egress.

13 Conclusion

This paper presented a natural-language-to-SQL business intelligence system built on a decoupled, layered architecture comprising a React 18 SPA, a FastAPI REST backend, a provider-agnostic LLM service layer, and hosted inference via Groq. The migration replaced three architectural elements simultaneously — the user interface, the backend's role, and the inference backend — while leaving the domain pipeline unchanged, ensuring that every observable behaviour difference is attributable to architectural rather than functional modification.

The browser became a pure consumer of typed JSON, holding no credentials and contacting no provider directly. FastAPI was promoted to the authoritative system boundary with declarative validation inbound, response models as a serialisation allow-list outbound, a uniform error envelope, and an asynchronous pipeline in which every blocking operation is explicitly offloaded. Local model serving was replaced by a two-method provider abstraction whose sole concrete implementation targets Groq, making the model vendor a configuration value rather than an architectural commitment.

The reasoning-suppression requirement motivated the most significant design work. The resulting mechanism — four independent layers comprising an API-level suppression parameter, a narrow-scope parser with no field for reasoning content, a subtractive sanitiser that fails closed, and response models that serialise only their declared fields — is robust by construction. The structural layers are the strongest: they cost nothing at runtime and cannot be bypassed by inputs the sanitiser did not anticipate.

Measured results confirm the intended operating characteristics: 472 ms mean model latency, 1,231 ms mean end-to-end latency, an 87% reduction in initial JavaScript payload, and clean typed error responses under live provider throttling. The costs are stated plainly: prompts leave the trust boundary, availability depends on a third party, and capacity is bounded by a contractual rate limit rather than by hardware. The provider abstraction ensures that these trade-offs remain reversible.

References

[1] T. Yu *et al.*, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," in *Proc. EMNLP*, 2018. [Online]. Available: https://arxiv.org/abs/1809.08887

[2] R. T. Fielding, "Architectural Styles and the De-

sign of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000. [Online]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

[3] S. Ramírez, "FastAPI Documentation." [Online]. Available: https://fastapi.tiangolo.com/

[4] Encode, "Starlette — The little ASGI framework." [Online]. Available: https://www.starlette.io/

[5] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP-IJCNLP*, 2019. [Online]. Available: https://arxiv.org/abs/1908.10084

[7] Chroma, "Chroma — the open-source embedding database." [Online]. Available: https://docs.trychroma.com/

[8] OpenAI, "Introducing gpt-oss." [Online]. Available: https://openai.com/index/introducing-gpt-oss/

[9] OpenAI, "OpenAI Harmony Response Format." [Online]. Available: https://cookbook.openai.com/articles/openai-harmony

[10] Groq, "Groq API Reference — Reasoning." [Online]. Available: https://console.groq.com/docs/reasoning

[11] Groq, "Groq API Reference — OpenAI Compatibility." [Online]. Available: https://console.groq.com/docs/openai

[12] Meta Open Source, "React Documentation." [Online]. Available: https://react.dev/

[13] E. You and the Vite team, "Vite — Next Generation Frontend Tooling." [Online]. Available: https://vite.dev/

[14] Microsoft, "TypeScript Documentation." [Online]. Available: https://www.typescriptlang.org/docs/

[15] Pydantic, "Pydantic v2 Documentation." [Online]. Available: https://docs.pydantic.dev/

[16] Encode, "HTTPX — A next-generation HTTP client for Python." [Online]. Available: https://www.python-httpx.org/

[17] SQLAlchemy, "SQLAlchemy 2.0 Documentation." [Online]. Available: https://docs.sqlalchemy.org/

[18] Plotly, "Plotly JavaScript Open Source Graphing Library." [Online]. Available: https://plotly.com/javascript/

[19] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications." [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applicat

[20] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, 2023. [Online]. Available: https://www.w3.org/TR/WCAG22/

[21] Mozilla, "Cross-Origin Resource Sharing (CORS) — MDN Web Docs." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS